

实验报告

数字逻辑与计算机组成实验

lab05: 移位寄存器与伪随机序列

姓名: 法童

学号: 251250046

邮箱: 251250046@smail.nju.edu.cn

实验时间: 2026-09-20

1 实验目的

1. 用 8 位移位寄存器加异或反馈实现 LFSR，得到周期为 $2^8 - 1 = 255$ 的伪随机序列。
2. 把消抖后的按键上升沿当作移位寄存器的时钟，做到按一次键正好走一步。
3. 用两位数码管以十六进制显示寄存器的当前值，在上板观察序列的变化。
4. 用仿真验证序列的周期、状态遍历性，以及全零这一退化状态的处理。

2 实验原理

LFSR 的次态由”移位”和”反馈”两部分决定：整串状态右移一位，空出来的最高位由若干抽头异或得到。本次取 $x_7x_6x_5x_4x_3x_2x_1x_0$ 的低四位参与反馈：

$$x_8 = x_4 \oplus x_3 \oplus x_2 \oplus x_0$$

把这条递推关系写成特征多项式，就是 $x^8 + x^4 + x^3 + x^2 + 1$ 。该多项式是本原多项式，所以从任一非零状态出发，寄存器会不重不漏地走完全部 $2^8 - 1 = 255$ 个非零状态再回到起点，序列周期为 255。

全零是这套结构唯一的不动点：反馈项 $0 \oplus 0 \oplus 0 \oplus 0 = 0$ ，移进来的还是 0，寄存器会永远停在 00000000。它不属于那 255 个状态所在的环，所以装载时必须显式处理。

按键不能直接当时钟。机械触点在按下和松开时都会抖动，一次按下可能产生几十个边沿，寄存器会一下子连走好几步。因此先用 10 ms 节拍对按键采样，连续两次都是 1 才认为真的按下了，再对这个稳定电平取上升沿，得到一个 100 MHz 时钟周期宽的脉冲——它既做消抖，又保证一次按下只产生一个时钟。

3 实验环境与器材

- 开发板：Nexys A7-100T (FPGA 型号 xc7a100tcs9324-1)
- 开发工具：Vivado 2020.2 (非工程模式, build.bat / sim.bat / prog.bat 脚本驱动)
- 硬件描述语言：Verilog HDL

4 设计思路

整个设计分三层：时基与按键处理 (tickgen、debounce)、伪随机序列本体 (lfsr)、显示 (hex7seg、display2)，最后由 top 接板级端口。

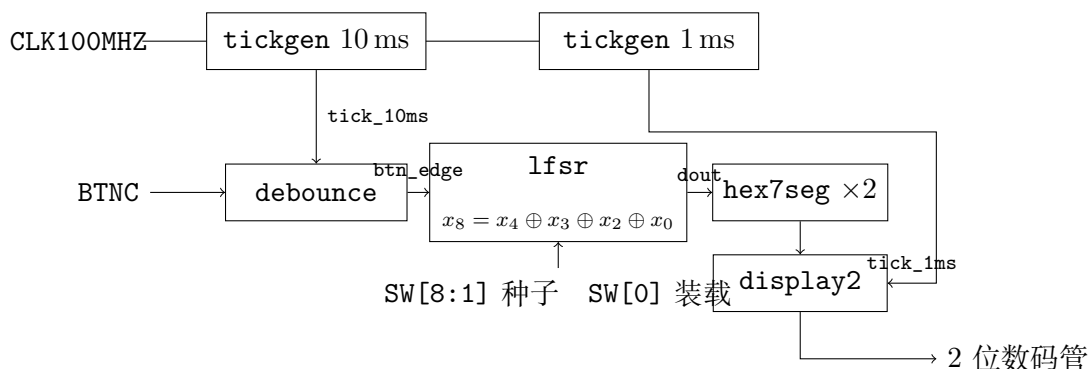


图 1: 系统框图

表 1: 板级资源分配

板级资源	顶层信号	功能
BTNC	clk	移位时钟（消抖后的按键上升沿）
SW[8:1]	seed	装载用的 8 位种子
SW[0]	load	装载使能，1 为装载、0 为移位
AN[1:0]、CA...CG	数码管	高四位 / 低四位的十六进制显示
AN[7:2]、DP	——	置 1 熄灭

核心模块只有三行有效逻辑：装载时取种子，否则右移一位并把反馈填进最高位。种子加了一个“全零替换成 01”的保护：

```

1 // 装载全零会锁死在不动点，这里把它替换成 01
2 wire [7:0] init = seed | {7'b0, ~|seed};
3
4 always @(posedge clk) begin
5     dout <= load ? init : {dout[4]^dout[3]^dout[2]^dout[0], dout
6         [7:1]};
7 end

```

top 把种子和装载开关接进核心，时钟则来自按键边沿：

```

1 lfsr u_lfsr(
2     .seed(SW[8:1]),

```

```

3      .clk(BTNC_edge),      //消抖后的按键上升沿
4      .load(SW[0]),
5      .dout(dout)
6  );

```

显示部分把 dout 拆成两个十六进制半字节交给 hex7seg 译码，再由 display2 用 1ms 节拍在两位数码管间轮扫（刷新率 500 Hz）。左边一位显高四位、右边一位显低四位，读数和寄存器的值一致，不用在脑子里对调。

5 实验过程

1. 编写 lfsr.v 与 display2.v, top.v 中时基复用 lab03 的 tickgen、消抖复用 debounce、译码复用 lab04 的 hex7seg。
2. 从根目录 nexysa7.xdc 复制约束，取消本次用到引脚的注释：CLK100MHZ、SW[8:0]、BTNC、AN[7:0]、CA..CG、DP。
3. 运行 build.bat lab05\lfsr 完成综合、实现并生成比特流，再 prog.bat lab05\lfsr 下载到板上。
4. 编写 sim/tb.v，用 sim.bat lab05\lfsr 跑仿真，导出 build/top.vcd 并查看波形。

```

671 Running DRC as a precondition to command write_bitstream
672 INFO: [IP_Flow 19-234] Refreshing IP repositories
673 INFO: [IP_Flow 19-1704] No user IP repositories specified
674 INFO: [IP_Flow 19-2311] Loaded Vivado IP repository 'D:/Xilinx/Vivado/2020.2/data/ip'.
675 INFO: [DRC 23-227] Running DRC with 2 threads.
676 WARNING: [DRC CFGDBVS-1] Missing CFGDBVS and CONFIG_VOLTAGE Design Properties: Neither the CFGDBVS nor CONFIG_VOLTAGE voltage property is set in the cur
677
678 set_property CFGDBVS value1 [current_design]
679 #Where value1 is either VCC0 or GND
680
681 set_property CONFIG_VOLTAGE value2 [current_design]
682 #Where value2 is the voltage provided to configuration bank 0
683
684 Refer to the device configuration user guide for more information.
685 WARNING: [DRC PDRC-153] Gated clock check: Net d_btnc/clk is a gated clock net sourced by a combinational pin d_btnc/dout[7]_i_2/O, cell d_btnc/dout[
686 u_lfsr/dout_reg[0], u_lfsr/dout_reg[1], u_lfsr/dout_reg[2], u_lfsr/dout_reg[3], u_lfsr/dout_reg[4], u_lfsr/dout_reg[5], u_lfsr/dout_reg[6], and u_lfs
687 INFO: [Vivado 12-3120] DRC finished with 0 Errors, 3 Warnings
688 INFO: [Vivado 12-3200] Please refer to the DRC report (report_drc) for more information.
689 INFO: [Designutils 20-2272] Running write_bitstream with 2 threads.
690
691 Loading data files...
692 Loading site data...
693 Loading route data...
694 Processing options...
695 Creating bitmap...
696 Creating bitstream...
697 Writing bitstream ./build/top.bit...
698 INFO: [Vivado 12-1842] Bitgen Completed Successfully.
699 INFO: [Project 1-120] WebTalk data collection is mandatory when using a WebPACK part without a full Vivado license. To see the specific WebTalk data
700 INFO: [Common 17-83] Releasing license: Implementation
701 11 Infos, 3 Warnings, 0 Critical Warnings and 0 Errors encountered.
702 write_bitstream completed successfully
703 write_bitstream: Time (s): cpu = 00:00:07 ; elapsed = 00:00:09 . Memory (MB): peak = 1988.879 ; gain = 382.266
704 INFO: [Common 17-200] Exiting Vivado at Sun Sep 20 22:34:12 2020...
705

```

图 2: 综合、实现与比特流生成完成

6 测试方法

如何验证：序列的周期和遍历性是”跑够长才知道”的性质，在板上按 255 次键不现实，所以这部分交给仿真，上板只验证与硬件相关的行为（装载、消抖、译码显示）。仿真里逐拍把 DUT 的状态和测试台自己重算的参考序列比对。

测试信号的选择：

- **手册示例序列：**种子取 01，看头五拍是否为 80、40、20、10、88。
- **从 01 跑满 255 拍：**检查 255 个状态两两不重复、中途不出现全零、第 255 拍回到 01。这三条合起来才说明周期确实是 255 而不是它的某个因数。
- **装载 00：**全零是退化状态，专门测它是否被保护逻辑替换成 01，而不是锁死。
- **换种子 A5：**换一个和 01 差得很远的初值，验证周期与种子无关。

上板测试：拨 SW[8:1] 给一个种子并按一次 BTNC 装载，看数码管是否立即显示该种子；之后连按按键，观察读数是否每按一次跳一次、松手不再变化（消抖生效），且读数看上去没有规律。

7 实验结果

7.1 仿真

```
--- case 1: manual example from 01 ---
    01 -> 80 -> 40 -> 20 -> 10 -> 88
--- case 2: full period from 01 ---
    covered 255 distinct nonzero states, back to 01 at step 255
--- case 3: all-zero seed ---
    all-zero seed starts from 01 and shifts normally
--- case 4: another nonzero seed A5 ---
    covered 255 distinct nonzero states, back to A5 at step 255

[PASS] all checks OK.
```

四条用例全部通过，说明设计满足”周期 255、遍历全部非零状态、全零被替换、周期与种子无关”这几点。

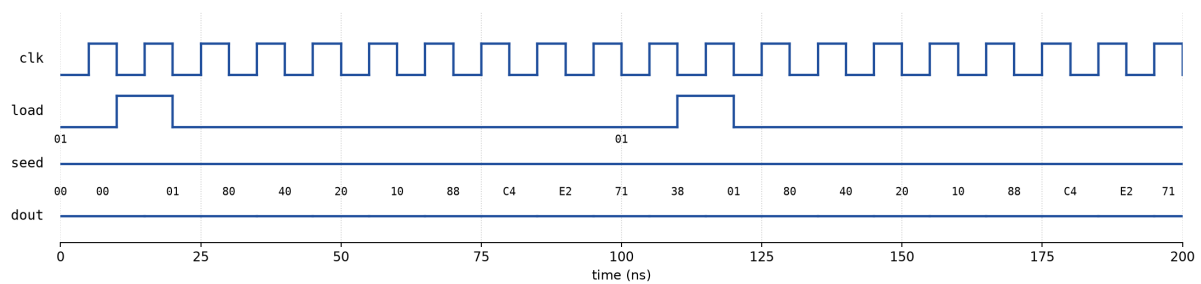


图 3: 仿真波形: 装载 01 后的手册示例序列 (dout 以十六进制标注)

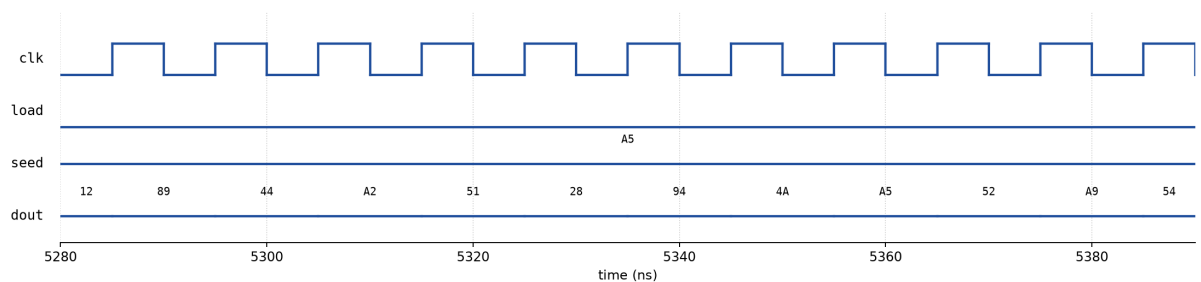


图 4: 仿真波形: 第 255 拍回到种子 A5

7.2 上板

上板后现象与设计一致: SW[8:1] 拨成某个种子、按一次 BTNC, 数码管立刻显示该种子的高、低两个半字节; 此后每按一次, 读数前进一格, 松手不再跳动; 连续按下时读数的变化看不出规律, 但每次按回同一个位置、装载同一个种子, 得到的序列完全相同——这正说明它是”伪”随机。

8 问题与解决办法

- **约束文件里多留了一个不存在的引脚:** 从 nexysa7.xdc 复制时把 SW[9] 也留了下来, 而 top 的端口只有 SW[8:0], 综合时报 WARNING: [Vivado 12-584] No ports matched 'SW[9]',紧接着是 CRITICAL WARNING: [Common 17-55] 'set_property' expects at least one object。原因是 get_ports 没匹配到任何对象, 属性无从设置。因为本次只用到 9 个开关, 把该行重新注释掉即可, 告警消失。教训是: 约束行必须和顶层端口一一对应, 多一行少一行都会报错。
- **装载全零会让序列锁死:** 手册明确要求”对全零状态进行特殊处理”, 最初的 `dout <= load ? seed : ...` 没有做这件事, 一旦种子拨成全零, 寄存器就永远停在 00。修法是给种子加一层保护 `init = seed | {7'b0, |seed}`: 种子非零时 init 就是种子本身, 全零时把最低位强制置 1, 退化成种子 01。



图 5: 上板实测：两位数码管显示当前寄存器值

9 思考题

生成的伪随机序列仍然有一定的规律，如何能够生成更加复杂的伪随机序列？

现有的规律来自两点：一是序列由一条线性递推式决定，只要知道 8 位状态和抽头位置就能往后推任意拍；二是序列的线性复杂度只有 8，用很少的测试位就能反推出整个生成器。对应的改进方向：

1. **加长寄存器**： n 位本原多项式对应的周期是 $2^n - 1$ ，把宽度从 8 提到 64，周期就从 255 涨到 $2^{64} - 1$ 量级，穷举和线性方程组都难以处理。练习里的那道 64 位 lfsr 就是这个思路。
2. **破坏线性**：反馈项里加入与项（非线性反馈移位寄存器 NFSR），或者用非线性函数对状态做滤波后再输出，线性复杂度就不再等于寄存器位数。Grain、Trivium 这类流密码都走这条路。
3. **输出方式**：不要每拍输出整个寄存器。本设计直接把 8 位状态送上数码管，相邻两拍之间明显的移位关系一眼就能看出；如果每拍输出原序列的某个特定排列、或输出状态的某个非线性函数，规律性会弱得多。

10 启示与建议

启示：

1. LFSR 用极小的代价换来了很长的序列：核心只是一个“移位 + 异或”的循环，但正因为状态空间有 255 个非零元素、而反馈恰好把整个空间串成一条链，输出才显得随机。用一个多项式把有限状态空间组织成了一整条环。
2. 全零这个不动点提醒我，反馈结构里的退化状态必须单独考虑。“随机”的前提是序列真的在变化。

建议：我阅读了本次实验手册之后，对 LFSR 通过一个“移位 + 异或”的简单算法，就能构造全部伪随机数序列的原理感到很好奇。因此我问了 AI，也查阅了一些百科，才慢慢搞懂这其中线性代数、特征多项式相关的种种新知识，在这个过程中我收获不少。如果实验手册在 LFSR 的原理上能稍作一些简明的讲解，或许能让大家学到更多。